

Legacy Banking Modernisation Platform — the gist

Everything you need to understand and present the project, in one short read.

Team StudyEdge · CII Eastern Region Hackathon · Banking and Finance Technology

Tamanna Panda · Swayam Subhankar Sahoo · Adyasha Das · Soumyajit Sarkar

Condensed from the full research dossier ([Legacy_Banking_Modernisation_Research_R2.pdf](#) , 97 pages) — go there for any depth this page skips.

1 · The idea in one sentence

Everyone proves the rewrite matches the legacy; we also ask whether the legacy matches the regulator — and we report the answer per business rule, including the rules we could not prove and the ones our own AI got wrong.

If you remember nothing else, remember that sentence. It is the approved one-line pitch, and every slide in the deck is an expansion of some clause of it.

2 · The problem, in sixty seconds

Banks run on COBOL programs written decades ago. The engineers who wrote them are gone, and **no specification was ever written** — the business rules exist only in the code. Translating COBOL to Java is a solved problem (IBM, AWS and TCS all ship tools for it). The unsolved problem is **proving the new program behaves identically to the old one**, because there is nothing written down to check it against. So the generated Java never reaches production, and banks rationally keep running software they cannot safely change.

Why it matters in India specifically: a one-paisa drift across millions of accounts is a *regulatory* event, not a bug report. In 2020 the RBI froze HDFC Bank's entire digital roadmap for over a year after repeated IT failures. In the UK, TSB's failed 2018 migration locked out customers for months and drew a £48.65m fine. That risk calculus — not laziness — is why the modernisation backlog is measured in decades.

3 · The insight that makes it ours

Every tool on the market compares two things: legacy output vs rewrite output, treating the legacy as correct *by definition*. Our key line:

“The legacy system is its own oracle” is true for equivalence and false for correctness. The gap between those two words is where a compliance finding lives — and no migration product is looking there.

A program written in 1987 can breach a Master Direction issued decades later, and a perfectly faithful migration carries the breach forward. So we run **three** things on the same input, not two: the legacy COBOL, the generated Java, and a small executable model (~60 lines, clause-cited) of what the RBI Master Direction actually prescribes — savings interest on a **daily product basis**, rupee-deposit interest **rounded to the nearest rupee**. Four outcomes fall out, and the valuable one — legacy and rewrite agree with each other while both breach the regulation — is structurally *invisible* to every two-way tool.

4 · What the system is

A deterministic pipeline with the LLM on a short leash (two bounded call sites, never the whole file):

1. **Intake gate** — programs that read the clock, hit databases or CICS are *refused with a reason*: non-determinism makes a differential verdict meaningless. Narrow scope is a design decision, not a limitation we hope nobody notices.
2. **Parse & slice** — ProLeap builds the AST; the program is cut into rule-sized slices.
3. **Extract rules** — one bounded LLM call per slice; line citations are written by the pipeline, not the model, so it cannot invent a reference to lines the slice doesn't contain.
4. **Rule-directed input generation** — the extracted rules say where the boundaries are, so the generator emits ₹99,999 / ₹1,00,000 / ₹1,00,001, accruals landing on exactly ₹0.50, 29 February, month ends.
5. **Execute and compare, three ways** — legacy (compiled with GnuCOBOL — the oracle), the untrusted Java rewrite, and the RBI reference model. Divergences are shrunk by delta debugging to one minimal counterexample.

Output: a verdict **per business rule** (proven / unproven / refuted), a mutation score saying how strong the whole claim is, compliance findings against the regulator, and a signed evidence bundle. Runs on a laptop — no mainframe, no production traffic, no licence fees.

5 · The three things nobody else does

Differential testing itself is **not** our invention — IBM, AWS and Mechanical Orchard all ship a version, and we say so proudly (it proves the diagnosis right). These three are ours:

Claim	What it means
1. We check the legacy against the regulator	The third reference (section 3 above). Only our tool can say “the migration is faithful <i>and</i> the books have been non-compliant for forty years.”
2. A verdict per business rule, not per test case	A rule counts as proven only when all three checks hold: its lines were <i>reached</i> ; inputs on both sides of its boundary produced <i>different</i> outputs (the boundary is actually live); and the rewrite <i>agreed</i> on all of them. The middle check is the clever one — it catches our own extraction layer inventing rules that are not in the program. When it fails, the verdict is refuted : our own AI, caught by our own harness, with a reproducible counterexample.
3. We test our own tests	“Everything passed” proves nothing unless the inputs could have caught a failure. So we break the legacy on purpose — move a threshold by one rupee, delete a ROUNDED — and check whether the corpus notices. Rules whose mutants survive get downgraded to unproven. The equivalence claim ships with its own strength score.

6 · The demo numbers — and how to say them

The illustrative output shape (for a savings-interest accrual program, 47 rules extracted):

31 proven · 11 unproven — named · 5 refuted — our AI was wrong · 94% mutation kill rate

Honesty note: these are the *illustrative* demo numbers used throughout the dossier and deck. Present them as “the shape of the tool’s output”, not as results from a completed production run.

The demo beats, in order (spoken lines in the dossier, Part 11): the COBOL (“this is the specification; there is no other one”) → the extracted rules (“a quarter of this is probably wrong, which is why this is not the product”) → the boundary corpus → **the divergence** (balance ₹1,00,000, 12 days at 3.5%: legacy ₹115, rewrite ₹116 — Java rounds HALF_UP, COBOL truncates) → **the refuted rule** (dormancy boundary our extraction invented; 23 and 25 months give identical output — this is the moment the room gets the thesis) → **the compliance finding** (both implementations round to two decimals; RBI says nearest rupee — this is the moment a banker starts caring) → the coverage table (why they trust it).

7 · Numbers to quote, numbers to avoid

Safe to quote (defensible)	Avoid (contested — will invite a fact-check)
92 of the world’s top 100 banks run on IBM mainframes; over 40% of banking systems still run COBOL	“95% of ATM swipes run on COBOL” — traces to one 2017 press item
BCG: Indian lenders must invest ~\$1bn over 5–10 years to upgrade legacy cores; Indian banks spend up to 5% of revenue on IT vs 7–9% globally	“800 billion lines of COBOL” — survey extrapolation; say “hundreds of billions” if pushed
TSB 2018: £48.65m FCA/PRA fine, £32.7m redress; RBI froze HDFC’s Digital 2.0 launches in Dec 2020, card freeze lifted Aug 2021	Precise COBOL-developer workforce numbers (retirement ages, headcounts) — direction is right, precision is not
Best published COBOL rule extractor: ~62% precision / 74% recall (A-COBREX) — quote it <i>against ourselves</i> : “we assume a quarter of our rules are wrong; that’s the design”	Any phrasing of “nobody does differential testing” or “we invented” — three vendors ship it
IBM Research on IBM’s own translation product: the output “cannot be trusted” — the strongest single line we have	“Everyone else’s submission assumes their AI is right” — true of the room, false of the market; use the one-sentence pitch instead

8 · The hard questions (short forms)

“**IBM / AWS already do this**” — Yes, and that convergence proves the diagnosis. Three differences: they need your mainframe or captured production traffic, we need only the source file; they report per test case, we report per business rule including the unproven ones; so we are usable at the *decision stage*, before a vendor or budget exists. Claim the deployment point and the unit of reporting — never the mechanism.

“**Your extraction is an LLM — how do you know the rules are right?**” — We don’t, and that’s the design. Extraction is a suggestion engine; execution is the judge. A rule is only *verified* when the differential run confirms it, and every rule carries line citations so a reviewer reads three lines, not four thousand.

“**GnuCOBOL isn't IBM Enterprise COBOL**” — The compiler is a dependency, not an assumption. GnuCOBOL (19 dialects, IBM-strict mode, 9,700 of 9,748 NIST tests) lets the demo run on a laptop; inside a bank you point the same harness at the bank's own compiler.

“**Interest accrual is a toy**” — Deliberately narrow: differential testing needs determinism, so we detect and refuse non-deterministic programs rather than mislabel them. And accrual is where the money is — a rounding drift there is a Master Direction breach across the whole deposit book.

“**Everything passed — what have you proved?**” — Equivalence over the inputs we ran, nothing more — which is why the mutation score exists and why unproven rules are named. Say “evidence of equivalence over the executed input set”, never “proof of correctness”.

“**Who buys it?**” — It precedes a migration engagement rather than competing with one: a bank deciding *whether* to modernise gets a specification and a regression suite for a system it currently can't touch — and keeps both even if it never migrates. An on-ramp, not a substitute.

9 · Business plan, in one breath (round-2 additions)

Investments: licence cost nil — GnuCOBOL, ProLeap, gcov, NIST suite and the AWS CardDemo corpus are all open. Prototype: 4 engineers × 12 weeks on commodity laptops; the only variable cost is bounded LLM inference. **Returns:** the ~\$1bn Indian core-upgrade spend is gated by exactly the assurance step we sell; the counterfactual is TSB 2018 (~£400m all-in). **Timeline:** working differential harness by week 2; the reciprocal loop and per-rule verdicts (the contribution) weeks 8–10; bank pilot months 4–9 inside the bank's own compiler environment. Team-authored sizing is flagged **[ASSUMPTION]** in the dossier — keep that honesty if asked.

10 · Where everything lives

If you need...	Go to
The full evidence for every claim above	Legacy_Banking_Modernisation_Research_R2.pdf (Parts 1–5)
The exact pitch wording at four lengths + demo script	Part 11 — Pitch Language
Judge Q&A, full answers	Part 5 — Risks and Judge Questions
The 10 diagrams (drop straight into slides)	Part 7 — Visuals, and Visuals/*.svg
The 13 deck slides as rendered	Part 15, and deck-renders/s01-13.png
Business-plan evidence and assumptions	Part 13 — Business Plan Research
Sources for every number	Part 16 — Sources

Close with the thesis, spoken plainly: “Of 47 rules extracted — 31 proven, 16 we could not prove, and here are the 16. Reporting what we didn't prove is what separates evidence from reassurance.”